

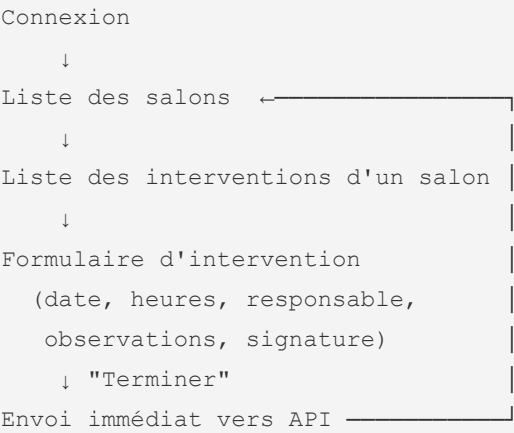
BVP — Documentation de l'application React Native

Application mobile de gestion d'interventions pour salons professionnels. Migration de Cordova/Preact vers React Native (Expo), réalisée en 2025.

1. Vue d'ensemble fonctionnelle

L'application BVP permet à des techniciens itinérants de gérer leurs interventions sur des salons professionnels depuis un téléphone Android.

Parcours utilisateur



Statuts d'une intervention

Code	Libellé	Signification
afaire	À faire	Intervention non encore réalisée
termineeatransferer	Terminée (en attente)	Formulaire rempli, envoi API en cours ou échoué
terminee	Terminée	Confirmée côté serveur

2. Stack technique

Outil	Version	Rôle
Expo SDK	54	Framework React Native
React Native	0.81.5	Moteur natif Android/iOS

Outil	Version	Rôle
React	19.1.0	UI déclarative
React Navigation v7	native-stack	Navigation entre écrans
expo-sqlite v16	API async/await	Base de données locale SQLite
react-native-signature-canvas	^5	Zone de signature tactile (via WebView)
@react-native-community/netinfo	11.4.1	Détection online/offline
Node.js	20 (via nvm)	Requis — Node 16 incompatible avec Expo 54

3. Architecture du projet

```

bvp-react-native/
├─ App.js           Point d'entrée, état global, navigation
├─ index.js         Enregistrement du composant racine
├─ app.json         Config Expo (nom, icônes, splash)
├─ dev-proxy.js     Proxy HTTP→HTTPS pour le dev (voir §7)
├─ .env             Variables d'environnement (non versionnées)
├─ src/
│   ├─ config/
│   │   ├─ env.js       Expose EXPO_PUBLIC_* en constantes JS
│   │   ├─ statuts.js   Mapping code statut → libellé + couleur
│   │   └─ structureIntervention.js Schéma attendu de l'API (validation)
│   ├─ api/
│   │   └─ api.js       Couche HTTP : fetch + headers auth
│   ├─ db/
│   │   └─ db.js        Toutes les opérations SQLite (CRUD)
│   ├─ lib/
│   │   ├─ log.js       Log console + insertion en DB
│   │   └─ synchroInterventions.js Synchronisation bidirectionnelle
│   ├─ components/
│   │   ├─ Header.js    Barre du haut (user, statut online, bouton sync)
│   │   └─ Breadcrumb.js Fil d'Ariane cliquable
│   └─ screens/
│       ├─ LoginScreen.js Connexion
│       ├─ SalonsScreen.js Liste des salons
│       ├─ InterventionsScreen.js Liste des interventions d'un salon
│       └─ InterventionScreen.js Formulaire + signature

```

Navigation (React Navigation v7)

```

App.js — NavigationContainer
├─ Stack.Navigator

```

```
|— Login      ← affiché si aucun user en DB
|— Salons     ← écran principal
|— Interventions (param: salonId)
|— Intervention (params: salonId, interventionId, salonNbInterventions)
```

Le header natif de React Navigation est désactivé (`headerShown: false`). Le `Header` maison est rendu directement dans `App.js` , au-dessus du `Stack.Navigator` , et n'apparaît que si un utilisateur est connecté.

4. Base de données locale (SQLite)

Fichier : `bvp.db` , géré via `expo-sqlite` v16 (API `async/await`).

Table `intervention`

Colonne	Type	Description
<code>id</code>	INTEGER PK	Clé locale (auto-incrémentée)
<code>roid</code>	INTEGER	ID côté serveur Symfony
<code>salon</code>	TEXT	Nom du salon
<code>salon_id</code>	TEXT	ID du salon côté serveur
<code>client</code>	TEXT	Nom du client
<code>date</code>	TEXT	Date ISO
<code>date_fr</code>	TEXT	Date au format JJ/MM/AAAA
<code>heure</code>	TEXT	Heure planifiée (HH:MM)
<code>type</code>	TEXT	Code du type d'intervention
<code>type_label</code>	TEXT	Libellé du type
<code>hall</code>	TEXT	Hall du salon
<code>stand</code>	TEXT	Stand
<code>contact_salon</code>	TEXT	Contact sur place
<code>telephone</code>	TEXT	Téléphone du contact
<code>precisions</code>	TEXT	Précisions sur l'intervention
<code>precisions_salon</code>	TEXT	Précisions sur le salon
<code>surveyjs_id</code>	TEXT	ID du questionnaire SurveyJS lié
<code>json_reponses</code>	TEXT	Réponses au formulaire (JSON)
<code>statut</code>	TEXT	<code>affaire</code> / <code>termineeatransferer</code> / <code>terminee</code>
<code>maj_local</code>	TEXT	Timestamp de la dernière modif locale

Colonne	Type	Description
<code>maj_remote</code>	TEXT	Timestamp de la dernière modif serveur

Table `user`

Un seul enregistrement à la fois (DELETE + INSERT à chaque login).

Colonne	Type	Description
<code>id</code>	INTEGER PK	Clé locale
<code>roid</code>	INTEGER	ID côté serveur (<code>user.id</code> retourné par l'API)
<code>username</code>	TEXT	Identifiant
<code>nom</code>	TEXT	Nom affiché
<code>date_connexion</code>	TEXT	Timestamp du dernier login

Attention : `roid` $\neq$ `id`. `roid` = l'identifiant côté Symfony, utilisé dans les appels API. `id` = l'identifiant SQLite local.

Table `log`

Colonne	Type	Description
<code>id</code>	INTEGER PK	—
<code>user_id</code>	INTEGER	ID local de l'utilisateur
<code>dthe</code>	TEXT	Timestamp ISO
<code>type</code>	TEXT	<code>info</code> ou <code>error</code>
<code>log</code>	TEXT	Message
<code>sync</code>	INTEGER	0 = non synchronisé côté serveur

5. API Symfony

Toutes les routes sont préfixées `/{{locale}}/api/` (ex : `/fr/api/`). L'authentification se fait via le header `Authorization: ApiKey <clé>`.

Méthode	Chemin	Description
POST	<code>login</code>	Retourne <code>{id, username, nom, date_connexion}</code>
GET	<code>interventions</code>	Liste toutes les interventions (filtrable par <code>salonId</code>)
POST	<code>set-interventions</code>	Envoie les interventions terminées (retourne HTTP 200 sans corps)

Méthode	Chemin	Description
POST	<code>set-log</code>	Envoie les logs applicatifs
GET	<code>surveyjs-config</code>	Retourne la configuration des questionnaires

Gestion des erreurs API (`api.js`)

La fonction `parseJson()` lit la réponse en texte brut avant de parser le JSON. Si l'API retourne du HTML (ex : Symfony mal configuré, mauvais `APP_ENV`), l'erreur affichée est : `HTTP <status> - réponse non-JSON : <début du HTML>` .

6. Synchronisation bidirectionnelle

Déclenchée automatiquement dans `App.js` dès que `isOnline && user` . Protégée par un flag `synchroEnCours` pour éviter les appels parallèles.

Étapes de la synchro (`synchroInterventions.js`)

1. ENVOI
Récupère en DB les interventions au statut "termineeatransferer"
→ POST /set-interventions
→ Si OK (pas d'exception) : passe ces interventions à "terminee" en DB
2. RÉCEPTION
GET /interventions → liste complète côté serveur
Vérifie la structure de chaque intervention reçue (guard clause)
3. COMPARAISON API ↔ DB
Pour chaque intervention API absente en DB → ajouter (INSERT)
Pour chaque intervention API terminée → supprimer si vieille de +5 jours
Pour chaque intervention DB absente de l'API → supprimer (sauf termineeatransferer)
4. LOG : "SYNCHRO :: END OK - créées: X, supprimées: Y"

Règle de conservation (5 jours)

Une intervention terminée n'est supprimée de la DB locale que si sa `maj_local` est antérieure à J-5. Cela permet de conserver un historique récent visible même hors ligne.

7. Description des écrans

LoginScreen

Formulaire identifiant / mot de passe. Appelle `POST /login`, stocke l'utilisateur en DB, puis relit depuis la DB pour garantir que `roid` est correct. Redirige vers `Salons` via `navigation.replace`.

SalonsScreen

Liste les salons via une requête `GROUP BY salon_id` en DB (chaque salon = agrégat des interventions qui lui appartiennent). Affiche le nom du salon et le nombre total d'interventions entre parenthèses.

Utilise `useFocusEffect` pour recharger la liste à chaque retour sur l'écran. Un second `useEffect` recharge aussi dès que la synchro se termine (évite la course critique : l'écran peut s'afficher avant la fin de la synchro).

InterventionsScreen

Liste les interventions d'un salon, **groupées par date** et **repliées par défaut** (comportement identique à l'app Cordova). Chaque groupe est un `SectionList` avec en-tête cliquable (▲/▼).

Chaque carte affiche : heure, numéro de référence (`roid`), client, type, hall, stand, contact, téléphone, précisions et badge de statut coloré.

InterventionScreen

Formulaire de saisie pour valider une intervention :

Champ	Obligatoire	Saisie
Date intervention	Non	JJ/MM/AAAA avec masque + clamping
Heure début	Oui	HH:MM avec masque + clamping
Heure fin	Oui	HH:MM avec masque + clamping
Nom du responsable	Oui	Texte libre
Observations	Non	Zone multilignes
Signature	Oui	Zone tactile (SignatureCanvas)

Flux de soumission : 1. Validation des champs obligatoires 2. `readSignature()` → déclenche `onOK` (signature présente) ou `onEmpty` (zone vide) 3. `onOK` → `soumettre()` : - Sauvegarde en DB locale (statut `termineeatransferer`) - Envoi immédiat à l'API (`POST /set-interventions`) - Si succès : passe à `terminee` en DB - Si échec : reste `termineeatransferer` pour la prochaine synchro 4. `navigation.goBack()`

La zone de signature utilise une WebView interne. Pendant le tracé, le scroll de la page est désactivé (`scrollEnabled = false`) pour éviter les conflits de gestes.

8. Environnement de développement

Architecture réseau locale

```
Téléphone Android (Expo Go)
  ↓ HTTP (pas de TLS)
172.17.176.216:8021 ← dev-proxy.js (écoute sur toutes les interfaces)
  ↓ HTTPS (certificat auto-signé ignoré)
127.0.0.1:8020      ← Symfony CLI (TLS activé par défaut)
```

Le proxy est nécessaire pour deux raisons : - Android refuse les certificats auto-signés générés par Symfony CLI -

`127.0.0.1` est le loopback de l'ordinateur, inaccessible depuis le téléphone

Commandes de démarrage (dans l'ordre)

```
# 1. Symfony - APP_ENV=bvp obligatoire (mauvaise DB sinon)
symfony server:stop
APP_ENV=bvp symfony server:start --port=8020 -d

# 2. Proxy HTTP → HTTPS
node /var/www/html/42morrowWebs/BVP/bvp-react-native/dev-proxy.js &

# 3. Expo - Node 20 obligatoire, --host lan pour l'IP réseau dans le QR code
cd /var/www/html/42morrowWebs/BVP/bvp-react-native
~/.npm/versions/node/v20.19.6/bin/npm expo start --host lan
```

URL à saisir dans Expo Go si le QR code n'est pas lisible : `exp://172.17.176.216:8081`

Variables d'environnement (`.env`)

```
EXPO_PUBLIC_API_ENDPOINT=http://172.17.176.216:8021/fr/api/
EXPO_PUBLIC_API_KEY=<clé API>
```

Attention : le chemin est `/fr/api/` (et non `/fr/`). Toutes les routes API sont sous `{locale}/api/` .

Vérification rapide du pipeline

```
curl -s -X POST http://172.17.176.216:8021/fr/api/login \
  -H "Content-Type: application/json" \
  -H "Authorization: ApiKey <clé>" \
  -d '{"username":"xxx","password":"yyy"}'
# Réponse attendue : {"id":..., "username":..., "nom":..., "date_connexion":...}
```

9. Roadmap (TODO)

Priorité haute

- **Formulaire dynamique par type d'intervention** : le formulaire actuel est figé (correspond au questionnaire `3504c4a3`). Les 3 autres questionnaires (`27b3e1d5` cubage, `48e0eae4` intervention simple, `28de79d5` logistique salon) ont des champs différents. L'API `surveyjs-config` retourne le JSON de configuration — à stocker en DB et lire dans `InterventionScreen` selon `intervention.surveyjs_id`.
- **Footer de debug** (comme dans l'app Cordova) : afficher en bas de chaque écran (hors Login) la date de build + 3 liens vers des écrans de debug : `LogScreen` (table `log`), `QueryScreen` (requête SQL libre), `DumpScreen` (comparaison DB ↔ API).

Priorité normale

- **Déconnexion** : bouton logout — vider la table `user` + reset de la navigation vers `Login` via `CommonActions.reset`.
- **Gestion offline explicite** : griser le bouton de sync et afficher un message si hors ligne (actuellement juste un point vert/rouge dans le Header).
- **Pull-to-refresh** sur `SalonsScreen` et `InterventionsScreen` : prop `refreshControl` sur `FlatList` / `SectionList` pour déclencher une synchro manuelle.

Priorité basse

- **Tests automatisés** : couvrir a minima `db.js` et `synchroInterventions.js`.
- **Build de production** : configurer `eas.json` pour générer un APK standalone (sans Expo Go). Nécessite un compte Expo.